

PRODUCT REQUIREMENTS DOCUMENT

PROMPTARA

Backend Service

Spring Boot · MySQL 8 · Flyway · Docker

AGENCY WORKSPACE PLATFORM

Kanban · Orders · Finance · Timeline · Team

Document Version: 1.0

Status: Draft for Implementation

Last Updated: 2026-06-06

Owner: PROMPTARA Engineering

Table of Contents

1.	Executive Summary	3
2.	Goals & Non-Goals	4
3.	Tech Stack & High-Level Architecture	5
4.	Non-Functional Requirements (NFR)	7
5.	Domain Model & Entity Relationships	10
6.	Database Schema (MySQL DDL)	12
7.	Flyway Migration Strategy	18
8.	API Specification — Authentication	20
9.	API Specification — Users & Team	24
10.	API Specification — Boards & Columns	26
11.	API Specification — Cards (Kanban Core)	29
12.	API Specification — Comments & Attachments	34
13.	API Specification — Notifications	36
14.	API Specification — Orders & Finance	38
15.	Concurrency Handling Strategy	41
16.	Low Latency & Performance Strategy	45
17.	Security Requirements	48
18.	Error Handling & Standard Response	51
19.	Observability — Logging, Metrics, Tracing	53
20.	Docker, Docker-Compose & Deployment	55
21.	Spring Boot Project Structure	58
22.	Implementation Roadmap & Milestones	60
23.	Appendix — Acceptance Criteria & Test Plan	62

1. Executive Summary

PROMPTARA adalah **agency workspace platform** yang mengelola seluruh siklus proyek desain kreatif — mulai dari brief masuk, eksekusi studio, preview klien, touch-up, hingga deal selesai (Done Deal). Frontend sudah berjalan di React 18 + Vite dengan state ter-persist di localStorage. Dokumen ini mendefinisikan backend yang akan menggantikan localStorage tersebut dengan persistensi server, kolaborasi real-time-ready, dan agregasi finansial untuk seluruh tim agency.

Backend ini dibangun di atas **Spring Boot 3.x (Java 21)**, **MySQL 8**, **Flyway** untuk version-controlled migration, dan dikemas dengan **Docker + docker-compose**. Sistem ditargetkan untuk traffic **multi-tenant per workspace**, dengan p95 latency < 200 ms pada endpoint hot path (board read, card drag-drop) dan dukungan concurrency control yang aman untuk operasi Kanban yang sering diedit bersamaan.

Key Modules

Module	Frontend View	Backend Responsibility
Auth	AuthScreen	Register, login, JWT issuance, refresh, logout
Board	BoardView (Kanban)	Board CRUD, column reorder, card drag-drop
Card	CardModal	Card CRUD, status transitions, financial fields
Comments	CardModal	Threaded comment per card, real-time fanout
Attachments	CardModal	File upload (S3-compatible), metadata, signed URL
Orders	OrdersView	Project history, filtering, CSV export
Finance	FinanceView	Aggregation, monthly summary, by-type/status
Calendar	CalendarView	Read-only timeline + month view from cards
Team	TeamView	Workspace member listing, role management
Notifications	NotifPanel	Due-soon, comment mention, assignment events

Why this matters

- **Persistence:** Data agency (revenue, klien, deadline) tidak boleh hilang saat ganti browser/device.
- **Kolaborasi:** Beberapa anggota tim membuka board yang sama secara bersamaan.
- **Audit:** Riwayat order dan finance perlu sumber data tunggal yang konsisten.
- **Skalabilitas:** Setiap workspace bisa punya 5-30 user, ratusan card aktif.
- **Keamanan:** Data klien & nilai project sensitif — harus per-user authorization.

2. Goals & Non-Goals

2.1 In-Scope (Goals)

- REST API JSON lengkap untuk semua TODO yang ada di frontend (auth, board, card, comment, attachment, notif, orders, finance).
- Auth berbasis **JWT access token (15 menit) + refresh token (7 hari)** dengan rotasi.
- **Role-based authorization**: Owner, Project Manager, Designer, Developer, Lainnya.
- **Multi-workspace per user** (1 user bisa join banyak workspace via membership).
- **Optimistic locking** pada card untuk drag-drop concurrent.
- **Soft-delete** pada entity penting (card, board) untuk audit & undo.
- **CSV / XLSX export** server-side untuk orders dan finance.
- **File upload** ke S3-compatible storage (MinIO untuk dev, AWS S3 untuk prod).
- **Notifikasi internal**: due-soon (cron), assignment (event), comment-mention.
- Docker-compose lokal yang menjalankan: API + MySQL + Redis + MinIO + Adminer.
- Flyway migration dengan strategi versioning yang aman untuk production.
- Observability: structured JSON logs, Prometheus metrics, health/readiness endpoints.

2.2 Out-of-Scope (Non-Goals)

- **Realtime WebSocket sync** — V1 cukup REST polling; WebSocket di V2.
- **Public API / third-party integration** (Slack, Trello sync) — V2.
- **Billing & subscription** (Stripe, Midtrans) — V2.
- **Mobile native app** — frontend web responsive sudah cukup di V1.
- **AI features** (auto-tagging, smart estimates) — terpisah dari core backend.
- **Multi-region replication** — single-region MySQL primary di V1.

2.3 Success Metrics

Metric	Target	Measurement
P50 latency (board read)	< 80 ms	Prometheus histogram
P95 latency (board read)	< 200 ms	Prometheus histogram
P95 latency (card update)	< 250 ms	Prometheus histogram
Error rate (5xx)	< 0.1 %	Daily aggregate
Availability (uptime)	99.5 % monthly	Healthcheck monitor

Metric	Target	Measurement
Cold start (Docker)	< 25 s	Container readiness probe
DB pool saturation	< 70 % steady-state	HikariCP metrics

3. Tech Stack & High-Level Architecture

3.1 Stack Summary

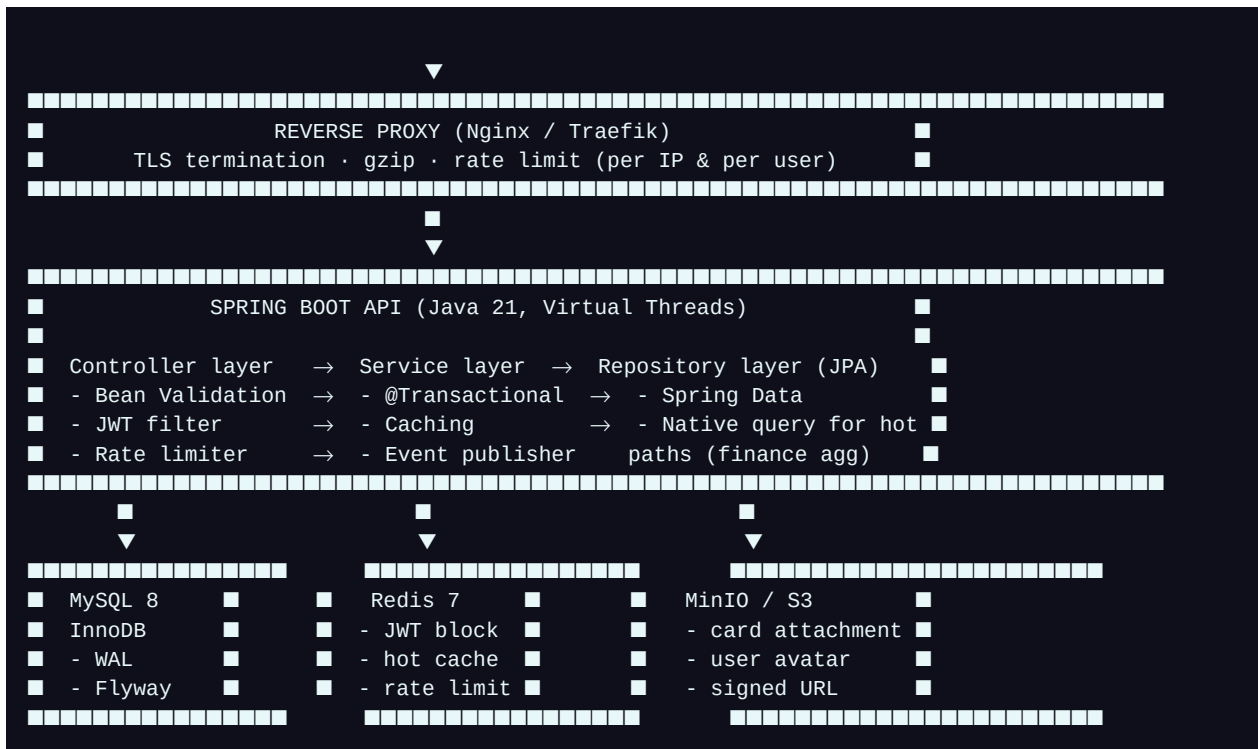
Layer	Technology	Justification
Language	Java 21 (LTS)	Virtual threads (Loom) untuk konkurensi murah
Framework	Spring Boot 3.3.x	Mature, ecosystem matang, Native AOT optional
Build	Maven 3.9 atau Gradle 8	Pilih salah satu, dokumen pakai Maven
Web	Spring Web MVC	Stable; Reactive (WebFlux) tidak diperlukan di V1
Security	Spring Security 6 + JWT	JWT issuance, BCrypt, method-level auth
Persistence	Spring Data JPA + Hibernate 6	ORM untuk model relasional kompleks
DB Driver	mysql-connector-j 8.4	Official MySQL connector
Database	MySQL 8.0 (InnoDB)	Row-level locking, MVCC, JSON column
Migration	Flyway 10.x	Version-controlled DDL, repeatable for seeds
Cache	Redis 7 + Spring Data Redis	Session blacklist, hot-data cache
File Storage	MinIO (dev) / S3 (prod)	Object storage untuk attachment
Validation	Jakarta Validation (Hibernate Validator)	Validasi pada DTO
JSON	Jackson 2.17	Default Spring serializer
Mapping	MapStruct 1.6	Compile-time DTO ↔ Entity mapper
Doc API	springdoc-openapi 2.x	Auto-generate OpenAPI 3 / Swagger UI
Test	JUnit 5, Mockito, Testcontainers	Unit + integration test
Container	Docker, docker-compose v2	Reproducible local & deployment
CI/CD	GitHub Actions	Build → Test → Push image → Deploy

3.2 High-Level Architecture

Architecture style

Classic **3-tier monolith** (Controller → Service → Repository) di-deploy sebagai single Spring Boot JAR di dalam Docker. Module-internal split per domain (*auth*, *board*, *card*, *finance*, *notification*). Tidak perlu microservices di V1 — over-engineering untuk skala saat ini.





3.3 Request Lifecycle (typical card update)

1. **Client** kirim PATCH /api/v1/cards/{id} dengan body partial update + header If-Match: <version>.
2. **Nginx** apply TLS, gzip, IP rate-limit, forward ke API.
3. **JWT filter** validasi access token; reject 401 jika invalid/expired.
4. **Rate limiter** per-user (Redis token bucket); reject 429 jika exceed.
5. **Controller** validasi DTO (Bean Validation).
6. **Service** load entity, cek version match, apply update di transaction.
7. **Repository** commit; Hibernate auto-increment version (optimistic locking).
8. **Event publisher** fire *CardUpdatedEvent* → notification consumer queue.
9. **Response** 200 dengan entity baru + version baru, atau 409 jika version mismatch.

4. Non-Functional Requirements (NFR)

4.1 Performance Targets

Endpoint Class	P50 Target	P95 Target	P99 Target	Rationale
Auth (login/refresh)	< 100 ms	< 250 ms	< 500 ms	BCrypt cost 12 dominan
Board read (cards list)	< 80 ms	< 200 ms	< 400 ms	Hot path; aggressive cache
Card create/update	< 100 ms	< 250 ms	< 500 ms	Tx + opt lock + audit
Card drag-drop	< 60 ms	< 150 ms	< 300 ms	Single column update
Comment add	< 80 ms	< 200 ms	< 400 ms	Insert + notif fanout async
Finance aggregate	< 200 ms	< 500 ms	< 1 s	Cached 30s; native query
Orders export CSV	< 1.5 s	< 3 s	< 5 s	Streaming response; not cached
File upload (5 MB)	< 2 s	< 4 s	< 8 s	Direct stream ke S3

4.2 Concurrency Requirements

- Backend harus aman jika **≥ 10 user** melakukan drag-drop di board yang sama bersamaan tanpa data corrupt.
- Operasi **reorder column** harus atomic (semua atau tidak sama sekali).
- **Counter** (cards.order, notification.unread_count) tidak boleh kehilangan increment.
- Komentar yang ditambahkan oleh dua user persis bersamaan harus dua-duanya tersimpan dengan urutan timestamp yang stabil.
- Refresh token rotation harus thread-safe — token lama langsung invalid setelah dipakai.
- **Login concurrent** dari device berbeda diperbolehkan; backend tidak invalidate sesi lain by default.

4.3 Scalability Targets

Dimension	V1 Target	V2 Target
Concurrent active users	200	2,000
Total registered users	5,000	50,000
Boards per workspace	100	500
Cards per board	500	2,000
Requests per second (steady)	100 RPS	1,000 RPS
Peak requests per second	300 RPS	3,000 RPS
DB pool size (HikariCP)	20 conn	50 + read replica

Dimension	V1 Target	V2 Target
Horizontal scaling	Single node	≥ 3 nodes behind LB

4.4 Availability & Reliability

- Uptime ≥ **99.5%** rolling 30 hari (max 3.6 jam downtime/bulan).
- **Graceful shutdown**: SIGTERM → tolak request baru → drain in-flight 30 detik → close DB pool.
- **Readiness probe** (/actuator/health/readiness) cek DB + Redis sebelum terima traffic.
- **Liveness probe** (/actuator/health/liveness) untuk restart jika deadlock.
- **Daily backup** MySQL (mysqldump + binlog) retensi 14 hari.
- **Point-in-time recovery** via binlog untuk insiden data corruption.

4.5 Maintainability

- Coverage ≥ **70%** di service layer (unit + integration).
- OpenAPI spec auto-generate dari controller — wajib up-to-date setiap PR.
- Migration Flyway **never edit** file yang sudah merged; selalu file baru.
- Naming convention: snake_case di DB, camelCase di Java/JSON.
- Semua API path versioned: /api/v1/. . . — breaking change wajib bump ke v2.

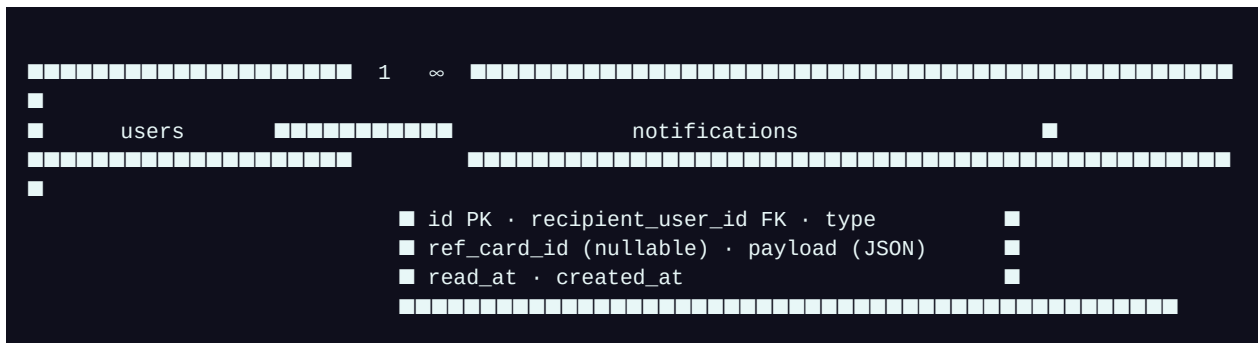
4.6 Compliance & Data

- Password **tidak pernah** dikembalikan di response apapun.
- Field sensitif (*password_hash*, *refresh_token*) tidak boleh ter-log.
- PII (email, nama) di-mask di structured log non-error level.
- Hak **delete account**: hard-delete user + anonimisasi referensi (replace with 'Deleted User').
- **Export semua data user** (GDPR-like) sebagai endpoint admin.

5. Domain Model & Entity Relationships

Berikut adalah ringkasan entity utama. Detail kolom & tipe ada di Bab 6 (DDL). Relasi digambarkan dengan notasi crow's foot sederhana.





5.1 Entity Summary

Entity	PK Type	Purpose
workspaces	UUID	Tenant root — agency container
users	UUID	Akun individu, login identity
workspace_members	BIGINT	Junction: user ↔ workspace dengan role
refresh_tokens	UUID	Server-side refresh token store (rotation)
boards	UUID	Kanban board per workspace (1 project bundle)
columns	UUID	Stage Kanban (Briefs, Studio, ...)
cards	UUID	Task/project utama, financial + assignment
card_assignees	BIGINT	Junction: card ↔ user
card_attachments	UUID	File metadata (object disimpan di S3)
comments	UUID	Diskusi per card
notifications	UUID	Event feed per user
audit_log	BIGINT	Append-only audit untuk perubahan sensitif

5.2 Identifier Choice — Why UUID v7?

- **UUID v7** (time-ordered) untuk PK utama: tidak bocorkan urutan ke client, tapi tetap sort-friendly di index B-tree.
- Hindari UUID v4 untuk PK karena random — fragmentasi index InnoDB tinggi.
- Junction table (assignees, members) tetap pakai BIGINT AUTO_INCREMENT — tidak diekspos ke client.
- Semua FK diberi index eksplisit; tidak mengandalkan implicit.

6. Database Schema (MySQL DDL)

Semua tabel pakai **InnoDB**, charset **utf8mb4**, collation **utf8mb4_0900_ai_ci**. Timestamp pakai **DATETIME(3)** (millisecond) UTC. ID pakai **CHAR(36)** (UUID v7 string form) untuk readability dan compatibility dengan Hibernate. Setiap tabel punya *composite index* untuk hot query path.

6.1 users

```
CREATE TABLE users (
  id          CHAR(36)      NOT NULL,
  email       VARCHAR(255)  NOT NULL,
  password_hash VARCHAR(72) NOT NULL,          -- BCrypt fixed length
  name        VARCHAR(120)  NOT NULL,
  role        VARCHAR(40)   NOT NULL,          -- Owner|PM|Designer|Developer|Other
  initials    VARCHAR(4)    NOT NULL,
  color       CHAR(7)       NOT NULL,          -- #RRGGBB
  email_verified TINYINT(1) NOT NULL DEFAULT 0,
  last_login_at DATETIME(3) NULL,
  created_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
              ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)  NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_email (email),
  KEY idx_users_deleted_at (deleted_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

6.2 workspaces & members

```
CREATE TABLE workspaces (
  id          CHAR(36)      NOT NULL,
  name        VARCHAR(120)  NOT NULL,
  slug        VARCHAR(60)   NOT NULL,
  owner_user_id CHAR(36)    NOT NULL,
  created_at   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)  NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_workspaces_slug (slug),
  KEY idx_workspaces_owner (owner_user_id),
  CONSTRAINT fk_ws_owner FOREIGN KEY (owner_user_id)
    REFERENCES users(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE workspace_members (
  id          BIGINT        NOT NULL AUTO_INCREMENT,
  workspace_id CHAR(36)     NOT NULL,
  user_id     CHAR(36)     NOT NULL,
  role        VARCHAR(40)   NOT NULL,          -- Owner|PM|Designer|Developer|Other
  joined_at   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_member (workspace_id, user_id),
  KEY idx_member_user (user_id),
  CONSTRAINT fk_member_ws FOREIGN KEY (workspace_id) REFERENCES workspaces(id)
    ON DELETE CASCADE,
  CONSTRAINT fk_member_usr FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3 refresh_tokens

```
CREATE TABLE refresh_tokens (  
  id          CHAR(36)      NOT NULL,  
  user_id     CHAR(36)      NOT NULL,  
  token_hash  CHAR(64)      NOT NULL,          -- SHA-256 of refresh JWT  
  user_agent  VARCHAR(255)  NULL,  
  ip_address  VARCHAR(45)   NULL,  
  expires_at  DATETIME(3)   NOT NULL,  
  revoked_at  DATETIME(3)   NULL,  
  created_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3),  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_refresh_hash (token_hash),  
  KEY idx_refresh_user (user_id, expires_at),  
  CONSTRAINT fk_refresh_user FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.4 boards

```
CREATE TABLE boards (  
  id          CHAR(36)      NOT NULL,  
  workspace_id CHAR(36)      NOT NULL,  
  name        VARCHAR(160)  NOT NULL,  
  client_name  VARCHAR(160)  NULL,  
  color       CHAR(7)       NOT NULL DEFAULT '#3EB8B8',  
  archived    TINYINT(1)    NOT NULL DEFAULT 0,  
  created_by  CHAR(36)      NOT NULL,  
  created_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3),  
  updated_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3)  
    ON UPDATE CURRENT_TIMESTAMP(3),  
  deleted_at  DATETIME(3)   NULL,  
  PRIMARY KEY (id),  
  KEY idx_board_workspace (workspace_id, deleted_at),  
  CONSTRAINT fk_board_ws FOREIGN KEY (workspace_id) REFERENCES workspaces(id),  
  CONSTRAINT fk_board_usr FOREIGN KEY (created_by) REFERENCES users(id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.5 columns

```
CREATE TABLE board_columns (  
  id          CHAR(36)      NOT NULL,  
  board_id    CHAR(36)      NOT NULL,  
  name        VARCHAR(80)   NOT NULL,  
  color       CHAR(7)       NOT NULL,  
  display_order INT         NOT NULL,  
  created_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3),  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_col_order (board_id, display_order),  
  KEY idx_col_board (board_id),  
  CONSTRAINT fk_col_board FOREIGN KEY (board_id) REFERENCES boards(id)  
    ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Naming

columns adalah reserved word di beberapa tool MySQL; pakai board_columns untuk aman.

6.6 cards

```
CREATE TABLE cards (
  id          CHAR(36)      NOT NULL,
  board_id    CHAR(36)      NOT NULL,
  column_id   CHAR(36)      NOT NULL,
  title       VARCHAR(255)  NOT NULL,
  description  TEXT          NULL,
  type        VARCHAR(30)   NOT NULL DEFAULT 'other', -- web-design|ui-ux|...
  priority    VARCHAR(10)   NOT NULL DEFAULT 'medium', -- high|medium|low
  client_name VARCHAR(160)   NULL,
  value_idr   BIGINT        NOT NULL DEFAULT 0,       -- nilai project
  dp_paid_idr BIGINT        NOT NULL DEFAULT 0,       -- DP diterima
  due_date    DATE          NULL,
  display_order INT         NOT NULL DEFAULT 0,       -- urutan di kolom
  version     BIGINT        NOT NULL DEFAULT 0,       -- @Version (optimistic lock)
  created_by  CHAR(36)      NOT NULL,
  created_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
                        ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3)   NULL,
  PRIMARY KEY (id),
  KEY idx_cards_board_col (board_id, column_id, display_order),
  KEY idx_cards_due       (due_date),
  KEY idx_cards_deleted   (deleted_at),
  KEY idx_cards_created   (created_at),
  CONSTRAINT fk_card_board FOREIGN KEY (board_id) REFERENCES boards(id),
  CONSTRAINT fk_card_col   FOREIGN KEY (column_id) REFERENCES board_columns(id),
  CONSTRAINT fk_card_user  FOREIGN KEY (created_by) REFERENCES users(id),
  CONSTRAINT chk_dp CHECK (dp_paid_idr >= 0 AND dp_paid_idr <= value_idr * 1.1)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.7 card_assignees

```
CREATE TABLE card_assignees (
  id          BIGINT      NOT NULL AUTO_INCREMENT,
  card_id     CHAR(36)    NOT NULL,
  user_id     CHAR(36)    NOT NULL,
  assigned_by CHAR(36)    NOT NULL,
  assigned_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_card_user (card_id, user_id),
  KEY idx_assignee_user (user_id),
  CONSTRAINT fk_assign_card FOREIGN KEY (card_id) REFERENCES cards(id)
    ON DELETE CASCADE,
  CONSTRAINT fk_assign_user FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.8 comments

```
CREATE TABLE comments (
  id          CHAR(36)      NOT NULL,
  card_id     CHAR(36)      NOT NULL,
  author_id   CHAR(36)      NOT NULL,
  body        TEXT          NOT NULL,
  created_at  DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  edited_at   DATETIME(3)   NULL,
  deleted_at  DATETIME(3)   NULL,
  PRIMARY KEY (id),
  KEY idx_cmt_card (card_id, created_at),
```

```
CONSTRAINT fk_cmt_card FOREIGN KEY (card_id) REFERENCES cards(id)
ON DELETE CASCADE,
CONSTRAINT fk_cmt_user FOREIGN KEY (author_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.9 card_attachments

```
CREATE TABLE card_attachments (
  id CHAR(36) NOT NULL,
  card_id CHAR(36) NOT NULL,
  uploaded_by CHAR(36) NOT NULL,
  filename VARCHAR(255) NOT NULL,
  mime_type VARCHAR(120) NOT NULL,
  size_bytes BIGINT NOT NULL,
  storage_key VARCHAR(500) NOT NULL, -- S3 object key
  checksum_sha256 CHAR(64) NULL,
  uploaded_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_att_card (card_id, uploaded_at),
  CONSTRAINT fk_att_card FOREIGN KEY (card_id) REFERENCES cards(id)
ON DELETE CASCADE,
  CONSTRAINT fk_att_user FOREIGN KEY (uploaded_by) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.10 notifications

```
CREATE TABLE notifications (
  id CHAR(36) NOT NULL,
  recipient_user_id CHAR(36) NOT NULL,
  type VARCHAR(40) NOT NULL, -- due-soon|comment|assignment|...
  message VARCHAR(500) NOT NULL,
  ref_card_id CHAR(36) NULL,
  ref_board_id CHAR(36) NULL,
  actor_user_id CHAR(36) NULL, -- who triggered it
  payload JSON NULL, -- flexible additional context
  read_at DATETIME(3) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_notif_user_unread (recipient_user_id, read_at, created_at),
  CONSTRAINT fk_notif_user FOREIGN KEY (recipient_user_id) REFERENCES users(id)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.11 audit_log

```
CREATE TABLE audit_log (
  id BIGINT NOT NULL AUTO_INCREMENT,
  actor_user_id CHAR(36) NULL,
  action VARCHAR(60) NOT NULL, -- CARD_CREATE|CARD_UPDATE|...
  entity_type VARCHAR(40) NOT NULL,
  entity_id CHAR(36) NOT NULL,
  before_json JSON NULL,
  after_json JSON NULL,
  ip_address VARCHAR(45) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_audit_entity (entity_type, entity_id, created_at),
  KEY idx_audit_actor (actor_user_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

```
PARTITION BY RANGE (TO_DAYS(created_at)) (  
  PARTITION p_2026q2 VALUES LESS THAN (TO_DAYS('2026-07-01')),  
  PARTITION p_2026q3 VALUES LESS THAN (TO_DAYS('2026-10-01')),  
  PARTITION p_max    VALUES LESS THAN MAXVALUE  
);
```

6.12 Index Strategy

Index	Why
idx_cards_board_col (board, column, display_order)	Hot path: load semua card di satu board,urut per kolom
idx_cards_due (due_date)	Cron due-soon scan, calendar view
idx_notif_user_unread (user, read_at, created_at)	Sidebar bell — list unread terbaru cepat
uq_col_order (board, display_order)	Mencegah dua kolom punya order sama di board sama
uq_users_email	Validasi register cepat + enforce uniqueness
uq_refresh_hash	Lookup refresh token saat /refresh

7. Flyway Migration Strategy

7.1 Directory Layout

```
src/main/resources/db/migration/
■■■ V1__init_users_and_workspaces.sql
■■■ V2__init_boards_columns_cards.sql
■■■ V3__init_comments_attachments.sql
■■■ V4__init_notifications_and_audit.sql
■■■ V5__seed_default_columns.sql
■■■ V6__add_card_priority_index.sql
■■■ ...
■■■ R__seed_demo_data.sql          (repeatable, dev/staging only)
```

7.2 Naming Convention

- **V<n>__** = versioned migration — **never** diubah setelah merged.
- **R__** = repeatable — di-run setiap kali checksum berubah; cocok untuk view, stored proc, seed dev.
- **U<n>__** = undo — **tidak dipakai di production**; lebih baik buat V baru yang revert.
- Nomor versi pakai **increment 1**, bukan timestamp — lebih mudah review PR.

7.3 Configuration (application.yml)

```
spring:
  flyway:
    enabled: true
    baseline-on-migrate: true
    baseline-version: 0
    locations: classpath:db/migration
    out-of-order: false          # strict order in production
    validate-on-migrate: true
    clean-disabled: true        # NEVER allow clean in any env from app
    table: flyway_schema_history
    placeholders:
      db_charset: utf8mb4
```

7.4 Migration Rules (Hard)

Aturan wajib

1. Setelah PR di-merge ke main, file **V<n>__** tidak boleh diubah. Perubahan checksum akan membuat Flyway gagal di production.
2. Setiap migration harus **backward-compatible** minimal untuk satu rilis. Contoh: tambah kolom → boleh; drop kolom → harus dua langkah (kosongkan dulu di rilis N, drop di rilis N+1).
3. Untuk perubahan besar (rename kolom, ubah tipe), pakai pola *expand* → *migrate* → *contract*.
4. Migration harus *idempotent on retry* sebisa mungkin: pakai `CREATE TABLE IF NOT EXISTS`, `ADD INDEX IF NOT EXISTS`.

7.5 Schema Change Patterns

Change	Pattern	Notes
Add column nullable	Single V file	Aman; default NULL
Add column NOT NULL	3-step expand/contract	Add nullable → backfill → set NOT NULL
Rename column	Add new → dual-write → drop old	Aplikasi tulis ke dua kolom 1 rilis
Drop column	Stop writing (rilis N) → drop (N+1)	Hindari window code mismatch
Add index large table	ALGORITHM=INPLACE, LOCK=NONE	MySQL 8 online DDL
Backfill data	Repeatable migration + batch	Chunk by PK range untuk hindari long tx

7.6 Production Deployment Flow

- **Pre-deploy:** CI run flyway info dan flyway validate terhadap staging snapshot.
- **Deploy:** Spring Boot startup → Flyway auto-migrate sebelum bean Hibernate init.
- **Failure:** Container gagal start → readiness probe fail → load balancer tidak route traffic → rilis aman di-rollback.
- **Rollback:** Jangan run undo; deploy ulang versi container lama (DB tetap forward).

8. API Specification — Authentication

Semua endpoint API berada di bawah prefix `/api/v1`. Body request/response menggunakan JSON UTF-8. Token dikirim lewat header `Authorization: Bearer <jwt>`.

8.1 Register

POST**`/api/v1/auth/register`**

Buat akun baru. Jika first user di workspace, otomatis Owner.

Auth: Public · **Role:** —

Request body

```
{
  "name": "Andi Pratama",
  "email": "andi@promptara.id",
  "password": "P@ssw0rd123!",
  "role": "Designer"
}
```

Validation rules

- name: 2-120 char, trimmed.
- email: format RFC 5322, lowercased server-side, unique.
- password: min 8, harus mengandung huruf + angka. Hash BCrypt cost 12.
- role: enum [Owner, Project Manager, Designer, Developer, Lainnya].

Response 201 Created

```
{
  "user": {
    "id": "01890d4c-7e0b-7c5e-9b3a-...",
    "name": "Andi Pratama",
    "email": "andi@promptara.id",
    "role": "Designer",
    "initials": "AP",
    "color": "#3EB8B8",
    "createdAt": "2026-06-06T08:11:23.482Z"
  },
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "expiresIn": 900
}
```

Error responses

Code	When	Body field
409 CONFLICT	Email sudah terdaftar	code: EMAIL_TAKEN
400 BAD REQUEST	Validation fail	fields[]: { name, message }
429 TOO MANY	Rate limit (5/menit per IP)	retryAfterSeconds

8.2 Login

POST**/api/v1/auth/login**

Tukar email+password jadi access+refresh token.

Auth: Public · Role: —

```
// Request
{ "email": "andi@promptara.id", "password": "P@ssw0rd123!" }

// Response 200
{ "user": { ... }, "accessToken": "...", "refreshToken": "...", "expiresIn": 900 }

// Response 401
{ "code": "INVALID_CREDENTIALS", "message": "Email atau password salah." }
```

Security hardening

Jangan beri tahu apakah email yang salah atau passwordnya. Pesan generic *'Email atau password salah'* mencegah user enumeration. Brute-force diatasi dengan rate limit + exponential backoff per email (5 attempt → lock 15 menit, di-track via Redis).

8.3 Refresh Token

POST**/api/v1/auth/refresh**

Tukar refresh token lama jadi access token + refresh token baru (rotation).

Auth: Refresh token · Role: —

```
// Request
{ "refreshToken": "eyJhbGciOi..." }

// Response 200 — NOTE: refreshToken adalah token BARU; yang lama langsung invalid
{ "accessToken": "...", "refreshToken": "...", "expiresIn": 900 }

// Response 401
{ "code": "REFRESH_TOKEN_REVOKED" }
```

Server behavior

- Hash refresh token (SHA-256) → lookup di refresh_tokens.
- Jika revoked_at IS NOT NULL atau expires_at < now() → 401.
- Mark old token revoked_at = now(), insert new row dengan token baru — **atomic dalam 1 transaction**.
- **Reuse detection:** Jika token yang sudah revoked dipakai lagi → revoke *seluruh chain* user tersebut (signal token theft).

8.4 Logout

POST**/api/v1/auth/logout**

Revoke refresh token saat ini + blacklist access token sampai expiry.

Auth: Bearer JWT · **Role:** Authenticated

```
// Request
{ "refreshToken": "eyJhbGciOi..." }

// Response 204 No Content
```

Access token JTI ditambahkan ke Redis dengan TTL = sisa lifetime token, sehingga filter JWT akan reject sampai expired natural.

8.5 Me (current user)

GET**/api/v1/auth/me**

Ambil profile user yang sedang login.

Auth: Bearer JWT · **Role:** Authenticated

```
// Response 200
{
  "id": "01890d4c-...",
  "name": "Andi Pratama",
  "email": "andi@promptara.id",
  "role": "Designer",
  "initials": "AP",
  "color": "#3EB8B8",
  "workspaces": [
    { "id": "...", "name": "PROMPTARA Studio", "role": "Designer" }
  ]
}
```

8.6 JWT Structure

```
// Access token (15 menit)
{
  "iss": "promptara-api",
  "sub": "01890d4c-7e0b-7c5e-9b3a-...", // userId
  "aud": "promptara-web",
  "exp": 1717661283,
  "iat": 1717660383,
  "jti": "9f1b...", // for blacklist
  "role": "Designer",
  "ws": "01890d4d-..." // active workspaceId
}

// Refresh token (7 hari) – disimpan hash-nya di DB
{
  "iss": "promptara-api",
  "sub": "01890d4c-...",
  "typ": "refresh",
  "jti": "rt-9f1b...",
  "exp": 1718265183
}
```

- Algoritma: **HS256** dengan secret rotasi setiap 90 hari (env var).

- Production siap pindah ke **RS256** jika perlu issuer/verifier split.
- Clock skew toleransi: 60 detik (untuk container time drift).

9. API Specification — Users & Team

9.1 List workspace members

GET `/api/v1/workspaces/{wsId}/members`

Daftar anggota workspace untuk Team view & assignee picker.

Auth: Bearer JWT · **Role:** Member

```
// Response 200
{
  "items": [
    {
      "id": "01890d4c-...",
      "name": "Andi Pratama",
      "email": "andi@promptara.id",
      "role": "Designer",
      "initials": "AP",
      "color": "#3EB8B8",
      "stats": { "totalAssigned": 12, "activeAssigned": 5 }
    }
  ],
  "total": 8
}
```

9.2 Invite member

POST `/api/v1/workspaces/{wsId}/members`

Undang user existing (by email) ke workspace.

Auth: Bearer JWT · **Role:** Owner / Project Manager

```
// Request
{ "email": "siti@promptara.id", "role": "Project Manager" }

// Response 201
{ "userId": "...", "role": "Project Manager", "joinedAt": "..." }
```

9.3 Update member role

PATCH `/api/v1/workspaces/{wsId}/members/{userId}`

Ubah role member (hanya Owner).

Auth: Bearer JWT · **Role:** Owner

```
// Request
{ "role": "Designer" }

// Response 200
{ "userId": "...", "role": "Designer" }
```

9.4 Remove member

DELETE `/api/v1/workspaces/{wsId}/members/{userId}`

Hapus dari workspace; assignment di-unassign cascade.

Auth: Bearer JWT · **Role:** Owner

Side effect

Saat member di-remove: semua **card_assignees** entry di-cascade delete. Card yang jadi *unassigned* akan generate notifikasi untuk Owner.

9.5 Update profile (self)

PATCH `/api/v1/users/me`

Update nama, color, password sendiri.

Auth: Bearer JWT · **Role:** Authenticated

```
// Request
{
  "name": "Andi P.",
  "color": "#EC4899",
  "currentPassword": "old...", // required jika ubah password
  "newPassword": "new..."    // optional
}

// Response 200 – user object terbaru
```

9.6 Delete account (self)

DELETE `/api/v1/users/me`

Hard delete account + anonimisasi referensi.

Auth: Bearer JWT · **Role:** Authenticated

- Owner workspace harus transfer kepemilikan dulu — kalau belum, response 409.
- Comments & audit log: `author_id` dipertahankan, tapi user tampil sebagai *'Deleted User'* via JOIN check.
- Refresh token di-revoke semua.

10. API Specification — Boards & Columns

10.1 List boards

GET `/api/v1/workspaces/{wsId}/boards`

Sidebar — list board aktif di workspace.

Auth: Bearer JWT · **Role:** Member

```
// Response 200
{
  "items": [
    {
      "id": "01890dab-...",
      "name": "Client Garuda Q3",
      "color": "#3EB8B8",
      "cardCount": 24,
      "createdAt": "2026-05-12T...",
      "archived": false
    }
  ],
  "total": 6
}
```

10.2 Create board

POST `/api/v1/workspaces/{wsId}/boards`

Buat board baru. Otomatis spawn 5 default column (Briefs, Studio, Preview, Touch Up, Done Deal).

Auth: Bearer JWT · **Role:** Owner / Project Manager

```
// Request
{ "name": "Brand X Rebranding", "clientName": "Brand X", "color": "#EC4899" }

// Response 201
{
  "id": "...",
  "name": "Brand X Rebranding",
  "color": "#EC4899",
  "columns": [
    { "id": "...", "name": "Briefs", "color": "#6366F1", "displayOrder": 0 },
    { "id": "...", "name": "Studio", "color": "#F59E0B", "displayOrder": 1 },
    { "id": "...", "name": "Preview", "color": "#3B82F6", "displayOrder": 2 },
    { "id": "...", "name": "Touch Up", "color": "#EC4899", "displayOrder": 3 },
    { "id": "...", "name": "Done Deal", "color": "#22C55E", "displayOrder": 4 }
  ]
}
```

10.3 Get board with columns + cards (hot path)

GET

/api/v1/boards/{boardId}

Load satu board lengkap untuk render Kanban view.

Auth: Bearer JWT · Role: Member

Performance critical

Endpoint ini adalah **hot path** — paling sering dipanggil. Implementasi: 1 query *boards*, 1 query *board_columns* (sorted), 1 query *cards* dengan JOIN FETCH ke assignees. Cache di Redis dengan key `board:{id}:v{updatedAt}` TTL 60 detik. Invalidate via *cache-aside* saat ada write.

```
// Response 200
{
  "id": "...",
  "name": "Brand X Rebranding",
  "color": "#EC4899",
  "columns": [
    {
      "id": "col-1", "name": "Briefs", "color": "#6366F1", "displayOrder": 0,
      "cards": [
        {
          "id": "card-...",
          "title": "Logo exploration round 1",
          "type": "branding",
          "priority": "high",
          "clientName": "Brand X",
          "valueIdr": 15000000,
          "dpPaidIdr": 7500000,
          "dueDate": "2026-06-30",
          "displayOrder": 0,
          "version": 4,
          "assigneeIds": ["user-...", "user-..."],
          "commentCount": 3,
          "attachmentCount": 2,
          "createdAt": "2026-06-01T..."
        }
      ]
    }
  ]
}
```

10.4 Update board

PATCH

/api/v1/boards/{boardId}

Rename, ubah warna, archive.

Auth: Bearer JWT · Role: Owner / Project Manager

10.5 Delete board (soft)

DELETE**/api/v1/boards/{boardId}**

Set deleted_at; card & column ikut tertutup. Bisa di-restore 30 hari.

Auth: Bearer JWT · **Role:** Owner

10.6 Reorder columns

PUT**/api/v1/boards/{boardId}/columns/reorder**

Atomic: kirim seluruh urutan baru.

Auth: Bearer JWT · **Role:** Owner / PM

```
// Request
{
  "order": ["col-3", "col-1", "col-2", "col-4", "col-5"]
}

// Response 200
{ "columns": [ ... ] }
```

Concurrency

Operasi reorder dijalankan dalam 1 transaksi dengan `SELECT ... FOR UPDATE` pada semua kolom board yang bersangkutan. Update `display_order` pakai trick *negate-then-flip* untuk hindari konflik unique key:

`UPDATE board_columns SET display_order = -display_order WHERE board_id=?`; kemudian set nilai positif baru.

10.7 Create column

POST**/api/v1/boards/{boardId}/columns**

Tambah stage Kanban kustom.

Auth: Bearer JWT · **Role:** Owner / PM

10.8 Rename / recolor column

PATCH**/api/v1/columns/{columnId}**

Update name dan/atau color.

Auth: Bearer JWT · **Role:** Owner / PM

10.9 Delete column

DELETE	/api/v1/columns/{columnId} Tolak jika ada card di dalamnya (force=true → pindah ke kolom pertama). Auth: Bearer JWT · Role: Owner
--------	---

11. API Specification — Cards (Kanban Core)

11.1 Create card

POST `/api/v1/boards/{boardId}/cards`

Buat card baru di kolom yang ditentukan.

Auth: Bearer JWT · **Role:** Member

```
// Request
{
  "columnId": "col-1",
  "title": "Logo exploration round 1",
  "description": "Eksplorasi mark + wordmark, minimal 3 opsi.",
  "type": "branding",
  "priority": "high",
  "clientName": "Brand X",
  "valueIdr": 15000000,
  "dpPaidIdr": 7500000,
  "dueDate": "2026-06-30",
  "assigneeIds": ["user-aa", "user-bb"]
}

// Response 201
{ "id": "card-...", "version": 0, ... } // full card object
```

Server side defaults

- displayOrder: server hitung $\text{MAX}(\text{displayOrder}) + 1024$ di kolom target (gap stride untuk reorder cepat).
- createdAt, updatedAt: server time UTC.
- createdBy: dari JWT sub.
- Validasi: $\text{dpPaidIdr} \leq \text{valueIdr} \times 1.1$ (DB constraint).

11.2 Update card (partial)

PATCH `/api/v1/cards/{cardId}`

Partial update. Semua field optional. Wajib kirim header If-Match.

Auth: Bearer JWT · **Role:** Member

```
// Headers
If-Match: 4 // current version, dari last GET

// Request (any subset)
{ "title": "Logo round 2", "priority": "medium", "dpPaidIdr": 10000000 }

// Response 200
{ "id": "card-...", "version": 5, ... }

// Response 409 (version conflict)
{
```

```
"code": "VERSION_CONFLICT",
"message": "Card sudah diubah orang lain. Refresh dulu.",
"currentVersion": 7
}
```

11.3 Move card (drag-drop) — atomic

POST**/api/v1/cards/{cardId}/move**

Endpoint khusus drag-drop. Lebih efisien dibanding PATCH umum.

Auth: Bearer JWT · **Role:** Member

```
// Request
{
  "toColumnId": "col-3",
  "toIndex": 2           // 0-based posisi di kolom target
}

// Response 200
{
  "id": "card-...",
  "columnId": "col-3",
  "displayOrder": 2560,   // server hitung di tengah dua tetangga
  "version": 6
}
```

Why a dedicated endpoint?

PATCH umum harus baca state penuh; /move bisa pakai **single UPDATE** tanpa load entity penuh. Hemat $\pm 40\%$ latency. Server menghitung displayOrder baru sebagai midpoint antara card di index-1 dan index — tidak perlu reorder ulang seluruh kolom. Setiap 50 move per board, background job *repack* spasi order.

11.4 Bulk reorder (rare path)

PUT**/api/v1/columns/{columnId}/cards/reorder**

Set ulang urutan seluruh card di satu kolom (jarang dipakai; backup path).

Auth: Bearer JWT · **Role:** Member

```
// Request
{ "cardIds": ["card-3", "card-1", "card-2"] }

// Response 200 — semua card dengan displayOrder baru (1024, 2048, 3072, ...)
```

11.5 Get card detail

GET**/api/v1/cards/{cardId}**

Detail penuh + comment + attachment list.

Auth: Bearer JWT · **Role:** Member

11.6 Delete card (soft)

DELETE**/api/v1/cards/{cardId}**

Set deleted_at; 30 hari di trash sebelum hard-delete cron.

Auth: Bearer JWT · **Role:** Owner / PM / Creator

11.7 Restore card

POST**/api/v1/cards/{cardId}/restore**

Undelete jika masih dalam 30 hari window.

Auth: Bearer JWT · **Role:** Owner / PM

11.8 List cards with filters (Orders View)

GET**/api/v1/workspaces/{wsId}/cards**

Filtering + sorting + pagination. Backbone untuk OrdersView.

Auth: Bearer JWT · **Role:** Member

Query params

Param	Type	Default	Notes
q	string	—	Search title, clientName (LIKE)
type	string	all	PROJECT_TYPES key
columnId	uuid	all	Filter by stage
boardId	uuid	all	Limit ke 1 board
priority	enum	all	high medium low
dueFrom	date	—	YYYY-MM-DD
dueTo	date	—	YYYY-MM-DD
sort	string	createdAt:desc	createdAt value dueDate :asc desc
page	int	1	1-based
size	int	20	Max 100

```
// Example
GET /api/v1/workspaces/ws-1/cards?q=brand&type=branding&page=1&size=20&sort=value:desc

// Response 200
{
  "items": [ { ... card summary ... } ],
  "page": 1, "size": 20, "total": 87, "totalPages": 5
}
```

11.9 Card events / activity

GET**/api/v1/cards/{cardId}/activity**

Riwayat perubahan dari audit_log untuk satu card.

Auth: Bearer JWT · **Role:** Member

12. API Specification — Comments & Attachments

12.1 List comments

GET**/api/v1/cards/{cardId}/comments**

Daftar komentar paling lama → terbaru. Pagination keyset.

Auth: Bearer JWT · **Role:** Member

```
// Query
?afterId=cm-12&size=30

// Response 200
{
  "items": [
    {
      "id": "cm-...",
      "author": { "id": "...", "name": "Andi", "initials": "AP", "color": "#..." },
      "body": "Sudah saya cek, looks good!",
      "createdAt": "2026-06-05T11:24:11.482Z",
      "updatedAt": null
    }
  ],
  "nextCursor": "cm-42"
}
```

12.2 Add comment

POST**/api/v1/cards/{cardId}/comments**

Buat komentar baru.

Auth: Bearer JWT · **Role:** Member

```
// Request
{ "body": "Sudah saya cek, looks good!" }

// Response 201 — comment object lengkap dengan timestamp server
```

Side effects

- Generate notifikasi tipe comment untuk semua assignee card kecuali author.
- Mention @nama → notifikasi tambahan ke user yang di-mention.
- Update cards.updated_at agar board cache invalid.

12.3 Edit / delete comment

PATCH**/api/v1/comments/{commentId}**

Author only. Set edited_at.

Auth: Bearer JWT · **Role:** Author**DELETE****/api/v1/comments/{commentId}**

Soft delete; body diganti '[deleted]' di response.

Auth: Bearer JWT · **Role:** Author / Owner

12.4 Upload attachment — presigned flow

Why presigned?

Upload file lewat backend = bandwidth + memory backend dipakai. Pakai pola **presigned URL**: backend cuma menerbitkan URL S3 berbatas waktu, client upload langsung ke S3. Backend tidak pernah handle byte file.

POST**/api/v1/cards/{cardId}/attachments/init**

Step 1: minta presigned URL untuk upload.

Auth: Bearer JWT · **Role:** Member

```
// Request
{ "filename": "mockup-v2.fig", "mimeType": "application/octet-stream", "sizeBytes": 8421500 }

// Response 200
{
  "attachmentId": "att-...",
  "uploadUrl": "https://s3.example.com/promptara-attach/...?X-Amz-Signature=...",
  "method": "PUT",
  "expiresIn": 900,
  "headers": { "Content-Type": "application/octet-stream" }
}
```

POST**/api/v1/attachments/{attachmentId}/complete**

Step 2: konfirmasi upload selesai; server verifikasi via HEAD ke S3.

Auth: Bearer JWT · **Role:** Member

```
// Request
{ "checksumSha256": "9af4..." } // optional, server verify

// Response 200
{
  "id": "att-...",
  "filename": "mockup-v2.fig",
  "sizeBytes": 8421500,
  "downloadUrl": "https://s3.example.com/...?X-Amz-Signature=...",
  "uploadedAt": "2026-06-06T..."
}
```

12.5 Download attachment

GET

/api/v1/attachments/{attachmentId}

Get metadata + presigned download URL (TTL 5 menit).

Auth: Bearer JWT · **Role:** Member

12.6 Delete attachment

DELETE

/api/v1/attachments/{attachmentId}

Hard delete row + delete object di S3 (idempotent).

Auth: Bearer JWT · **Role:** Uploader / Owner / PM

Limits

Max size: 50 MB per file (matches frontend hint).**Allowed types:** image/*, application/pdf, application/figma, application/illustrator, octet-stream untuk .fig/.ai.**Total quota:** 5 GB per workspace di V1 (configurable per workspace di V2).

13. API Specification — Notifications

13.1 List notifications

GET `/api/v1/notifications`

Sidebar bell — list notifikasi user, terbaru duluan.

Auth: Bearer JWT · **Role:** Authenticated

```
// Query
?unreadOnly=true&size=20&beforeId=ntf-42

// Response 200
{
  "items": [
    {
      "id": "ntf-...",
      "type": "due-soon",
      "message": "Logo round 2 akan jatuh tempo besok.",
      "refCardId": "card-...",
      "refBoardId": "board-...",
      "actorUser": null,
      "read": false,
      "createdAt": "2026-06-06T07:00:00.000Z"
    }
  ],
  "unreadCount": 7,
  "nextCursor": "ntf-22"
}
```

13.2 Mark one as read

POST `/api/v1/notifications/{id}/read`

Idempotent.

Auth: Bearer JWT · **Role:** Authenticated

13.3 Mark all as read

POST `/api/v1/notifications/read-all`

Set `read_at = now()` untuk semua notif user yang masih unread.

Auth: Bearer JWT · **Role:** Authenticated

13.4 Notification types

Type	Trigger	Recipient
due-soon	Cron daily 07:00 WIB scan card due dalam ≤ 3 hari	Semua assignee
overdue	Cron daily; card melewati dueDate dan belum Done	Semua assignee + Owner
assignment	Card assignees berubah	User yang baru ditambahkan
comment	Komentar baru di card	Assignee + author komentar sebelumnya
mention	@nama di body komentar	User yang di-mention
status-changed	Card pindah ke Done Deal	Owner workspace
dp-received	dpPaidIdr berubah ke $\geq$ valueldr	Owner workspace

13.5 Delivery pipeline

```
Service writes card → publish DomainEvent (Spring ApplicationEventPublisher)
                    → @TransactionalEventListener (AFTER_COMMIT)
                    → NotificationFanoutService
                      ■■ insert notifications rows (batch insert)
                      ■■ publish ke Redis pub-sub channel "notif:{userId}"
                      ■■ (V2) push WebSocket frame ke connected sessions

// V1: client poll GET /notifications setiap 30 detik kalau tab focused
```

14. API Specification — Orders & Finance

Orders dan Finance adalah **read-side projection** dari tabel *cards*. Tidak ada tabel terpisah — cukup query agregasi dengan index yang tepat dan cache.

14.1 Orders summary (band atas)

GET `/api/v1/workspaces/{wsId}/orders/summary`

Total nilai, diterima, outstanding, jumlah project.

Auth: Bearer JWT · **Role:** Member

```
// Response 200 (cached 30 detik)
{
  "totalValueIdr": 142500000,
  "totalReceivedIdr": 87500000,
  "totalOutstandingIdr": 55000000,
  "totalOrders": 38
}
```

14.2 Orders list (table view)

Reuses GET `/workspaces/{wsId}/cards` dari Bab 11.8 dengan filter & sort yang sama.

14.3 Export orders to CSV

GET `/api/v1/workspaces/{wsId}/orders/export`

Streaming CSV (Content-Type: text/csv; charset=utf-8).

Auth: Bearer JWT · **Role:** Member

```
// Query: sama dengan GET /cards (q, type, columnId, sort, ...)
// Response 200 - streaming, mendukung filter besar tanpa load semua ke memori

Content-Type: text/csv; charset=utf-8
Content-Disposition: attachment; filename="promptara-orders-2026-06-06.csv"

Tanggal Order,Klien,Judul Project,Tipe,Status,Nilai (IDR),DP (IDR),Sisa (IDR),Deadline
"06 Jun 2026","Brand X","Logo round 2","Branding","Studio",15000000,7500000,7500000,"30 Jun 20
26"
...
```

Streaming

Pakai `StreamingResponseBody` + JDBC cursor untuk fetch per 500 row. Tidak load seluruh result set ke memori. CSV header pakai BOM (`\uFEFF`) supaya buka di Excel ID langsung benar.

14.4 Finance summary (Finance View)

GET

/api/v1/workspaces/{wsId}/finance/summary

Semua agregat finance dalam 1 response.

Auth: Bearer JWT · Role: Member

```
// Response 200 (cached 60 detik di Redis, key per workspaceId)
{
  "totals": {
    "valueIdr": 142500000,
    "receivedIdr": 87500000,
    "outstandingIdr": 55000000,
    "doneRevenueIdr": 62000000
  },
  "monthly": [
    { "month": "2026-01", "label": "Jan 26", "valueIdr": 18000000, "receivedIdr": 12000000, "count": 4 },
    { "month": "2026-02", "label": "Feb 26", "valueIdr": 22500000, "receivedIdr": 22500000, "count": 6 },
    ...
  ],
  "byType": [
    { "type": "branding", "valueIdr": 45000000, "count": 9 },
    { "type": "web-design", "valueIdr": 38000000, "count": 7 },
    ...
  ],
  "byStatus": [
    { "columnId": "col1", "columnName": "Briefs", "valueIdr": 12000000, "count": 4 },
    ...
  ],
  "recent": [ { "id": "card-...", "title": "...", "valueIdr": ..., "dpPaidIdr": ..., "columnId": "..." } ]
}
```

SOL pattern (native query)

```
-- Monthly (driven by index idx_cards_created)
SELECT DATE_FORMAT(created_at, '%Y-%m') AS ym,
       SUM(value_idr) AS total_value,
       SUM(dp_paid_idr) AS total_received,
       COUNT(*) AS count
FROM cards
WHERE board_id IN (SELECT id FROM boards WHERE workspace_id = ?)
  AND deleted_at IS NULL
  AND value_idr > 0
  AND created_at >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
GROUP BY ym
ORDER BY ym;

-- By type, by status: similar pattern; satu query masing-masing
```

14.5 Export finance

GET

/api/v1/workspaces/{wsId}/finance/export

CSV summary bulanan.

Auth: Bearer JWT · Role: Member

14.6 Finance audit endpoint (Owner only)

GET	/api/v1/workspaces/{wsId}/finance/dp-changes Histori perubahan DP (siapa, kapan, berapa). Auth: Bearer JWT · Role: Owner
-----	--

15. Concurrency Handling Strategy

Kanban adalah aplikasi **collaborative**. Concurrency adalah *first-class concern*, bukan *afterthought*. Dokumen ini menjabarkan strategi per skenario.

15.1 Optimistic Locking (default untuk cards)

Setiap card punya kolom version BIGINT. Hibernate @Version akan otomatis menambahkan AND version = ? pada WHERE saat UPDATE. Jika 0 row affected → throw OptimisticLockException. Controller translate ke **HTTP 409** dengan currentVersion agar client bisa merge.

```
@Entity @Table(name = "cards")
public class CardEntity {
    @Id @Column(length = 36) private String id;
    @Version private Long version;
    // ... other fields
}

// In service layer
@Transactional
public CardDto update(String id, CardUpdateRequest req, long ifMatchVersion) {
    CardEntity c = cardRepo.findById(id).orElseThrow(NotFound::new);
    if (c.getVersion() != ifMatchVersion) {
        throw new VersionMismatchException(c.getVersion());
    }
    mapper.applyUpdate(req, c); // partial copy
    return mapper.toDto(c);     // flush: WHERE id=? AND version=?
}
```

15.2 Pessimistic Locking (untuk reorder column / card)

Reorder = banyak row diubah *display_order*-nya, harus konsisten. Pakai SELECT ... FOR UPDATE untuk lock baris yang akan disentuh, dalam 1 transaksi.

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void reorderColumns(UUID boardId, List<UUID> orderedIds) {
    // 1. Lock semua row board ini (acquire X lock)
    List<ColumnEntity> cols = columnRepo.lockByBoardId(boardId);
    // 2. Validate ID set match
    if (!sameIdSet(cols, orderedIds))
        throw new BadRequest("ID mismatch");
    // 3. Negate trick – hindari unique key collision sementara
    columnRepo.negateOrders(boardId);
    // 4. Assign nilai baru
    for (int i = 0; i < orderedIds.size(); i++) {
        columnRepo.setOrder(orderedIds.get(i), i);
    }
}

// Repository
@Query(value = "SELECT * FROM board_columns WHERE board_id = :id FOR UPDATE",
        nativeQuery = true)
List<ColumnEntity> lockByBoardId(@Param("id") UUID id);
```

15.3 Card Drag-Drop — Sparse Order Strategy

Tradisi: setiap card punya integer order rapat (1,2,3,...). Masalah: drag-drop bisa men-trigger update banyak row.

Solusi: **sparse / fractional ordering**. Saat insert card baru, `displayOrder` = MAX + 1024. Saat move ke tengah, `displayOrder` = (prev + next) / 2. Update hanya **1 row**. Tanpa perlu lock semua kolom.

```
// Service untuk /cards/{id}/move
@Transactional
public CardDto move(UUID cardId, UUID toCol, int toIndex) {
    CardEntity c = cardRepo.findByIdForUpdate(cardId)
        .orElseThrow(NotFound::new);
    List<Long> orders = cardRepo.findOrdersByColumn(toCol); // already sorted
    long newOrder;
    if (orders.isEmpty()) {
        newOrder = 1024L;
    } else if (toIndex <= 0) {
        newOrder = orders.get(0) - 1024;
    } else if (toIndex >= orders.size()) {
        newOrder = orders.get(orders.size() - 1) + 1024;
    } else {
        newOrder = (orders.get(toIndex - 1) + orders.get(toIndex)) / 2;
    }
    c.setColumnId(toCol);
    c.setDisplayOrder(newOrder);
    return mapper.toDto(c); // single UPDATE
}

// Periodic compaction (cron weekly): repack jika ada gap < 4
// UPDATE cards SET display_order = ROW_NUMBER() OVER (...) * 1024 WHERE column_id = ?
```

Edge case

Jika gap antara dua tetangga = 1 (sangat jarang), insert akan gagal hitung midpoint. Trigger background **repack** kolom tersebut secara on-demand, kemudian retry move sekali.

15.4 Comment Append — Insert-Only (Lock-Free)

Comment cuma insert; tidak ada race condition selama `created_at` + `id` (UUID v7) sebagai composite sort. Dua comment di millisecond yang sama tetap punya UUID v7 berbeda — urutan deterministic.

15.5 Refresh Token Rotation — Atomic Compare-and-Swap

Bisa terjadi: client kirim refresh request 2x bersamaan (network retry). Strategi:

- Token disimpan sebagai **hash** di DB (`token_hash` UNIQUE).
- Refresh op pakai UPDATE `refresh_tokens` SET `revoked_at` = NOW(3) WHERE `token_hash` = ? AND `revoked_at` IS NULL.
- Cek `rowsAffected` == 1. Jika 0 → token sudah revoked → reject 401 + **kill seluruh chain user** (signal possible theft).
- Hanya request yang sukses CAS yang dapat token baru. Yang kalah dapat 401.

15.6 Counter — `notifications.unreadCount`

Tidak di-cache sebagai kolom; hitung selalu via `COUNT(*) WHERE read_at IS NULL` dengan index `idx_notif_user_unread`. Cepat (≤ 5 ms) selama unread < 10k. Lebih simple dan zero drift.

15.7 Transaction Boundaries

Operation	Isolation	Why
Card create/update	READ_COMMITTED	Default; cukup dengan @Version
Column reorder	READ_COMMITTED + FOR UPDATE	Eksplicit lock subset row
Refresh token rotation	READ_COMMITTED	CAS lewat UPDATE WHERE
Finance summary read	READ_COMMITTED (read-only)	Tolerate slight drift
Soft delete card	READ_COMMITTED	Single UPDATE
Workspace owner transfer	REPEATABLE_READ	Cek 2 row sebelum swap

15.8 Deadlock Prevention

- Selalu acquire lock dengan **urutan ID konsisten** (ascending). Mis. saat update 2 card → lock card dengan id lebih kecil dulu.
- Set timeout: `innodb_lock_wait_timeout = 5` (detik) di MySQL. Bukan default 50 — fail fast lebih baik daripada thread stuck.
- Translate 1213 `ER_LOCK_DEADLOCK` → automatic retry 1x dengan backoff 50 ms.

16. Low Latency & Performance Strategy

16.1 Tomcat / Virtual Threads

```
# application.yml
spring:
  threads:
    virtual:
      enabled: true      # Java 21 virtual threads – cheap per-request
server:
  tomcat:
    threads:
      max: 200           # platform threads pool
      min-spare: 20
    accept-count: 200
    connection-timeout: 5s
    max-connections: 8192
  compression:
    enabled: true
    mime-types: application/json, text/csv, text/plain
    min-response-size: 1024
```

Virtual threads win

Workload Spring Boot kita banyak *blocking IO* (JDBC, S3, Redis). Dengan virtual threads, kita bisa handle ribuan request concurrent tanpa kehabisan platform thread. Tidak perlu rewrite ke WebFlux.

16.2 Database Connection Pool (HikariCP)

```
spring:
  datasource:
    hikari:
      pool-name: promptara-hikari
      maximum-pool-size: 20      # Java 21 + virtual threads tetap perlu cap koneksi DB
      minimum-idle: 5
      connection-timeout: 3000   # 3 s – fail fast
      idle-timeout: 600000       # 10 min
      max-lifetime: 1800000      # 30 min – rotate sebelum MySQL wait_timeout
      keepalive-time: 300000     # 5 min
      leak-detection-threshold: 30000
      data-source-properties:
        cachePrepStmts: true
        prepStmtCacheSize: 250
        prepStmtCacheSqlLimit: 2048
        useServerPrepStmts: true
        rewriteBatchedStatements: true
```

Rumus pool size: $\text{connections} = ((\text{core_count} \times 2) + \text{effective_spindle_count})$. Untuk single VM 4 vCPU + SSD: ~10-20 cukup. Lebih banyak $\neq$ lebih cepat; malah memperburuk lock contention di MySQL.

16.3 Caching Layer (Redis)

Key Pattern	TTL	Invalidation
board:{id}:full	60 s	Manual on card/column write
finance:{wsId}:summary	30 s	TTL only — drift acceptable
user:{id}:profile	300 s	On profile update
workspace:{id}:members	120 s	On member add/remove
jwt:blacklist:{jti}	= remaining JWT TTL	Self-expire
ratelimit:{userId}:{bucket}	60 s	Token bucket refill
login:fail:{email}	900 s (15 min)	Reset on success

```
@Cacheable(value = "board", key = "#boardId")
public BoardDto getBoard(UUID boardId) { ... }

@CacheEvict(value = "board", key = "#boardId")
public void updateBoard(UUID boardId, ...) { ... }

// Tipe cache: cache-aside (read), write-through (eviction). Tidak pakai write-back.
```

16.4 Query-Level Optimizations

- **N+1 prevention:** JOIN FETCH untuk relasi yang selalu di-load bersama (card → assignees). Entity Graph untuk yang opsional.
- **Projection DTO:** jangan return entity penuh kalau hanya beberapa kolom yang dipakai. Pakai interface projection Spring Data.
- **Native query** untuk finance aggregation (kompleks, perf critical).
- **Batch fetch:** `spring.jpa.properties.hibernate.default_batch_fetch_size: 32`.
- **Statement cache:** Hikari + MySQL serverside prepared statement.
- **Pagination keyset** (cursor-based) untuk daftar panjang — hindari OFFSET besar.

16.5 Response Size Reduction

- Jackson: `@JsonInclude(NON_NULL)` + `WRITE_DATES_AS_TIMESTAMPS=false`.
- Field sensitif (passwordHash, refreshToken) di entity → `@JsonIgnore`.
- Gzip dinyalakan di Spring Boot untuk respons ≥ 1 KB.
- Endpoint board read: *flattened* — assignee cukup ID, full user object di-fetch terpisah dan di-cache.

16.6 Async / Background Tasks

```
@Configuration @EnableAsync
public class AsyncConfig {
    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        return Executors.newThreadPerTaskExecutor(
            Thread.ofVirtual().name("promptara-async-", 0).factory()
        );
    }
}
```

```
    }  
  }  
  
  // Use cases:  
  // - Notification fanout (sehabis card move → notify assignees)  
  // - Email send (async, never block request)  
  // - S3 cleanup (delete orphan attachments)  
  // - Audit log write (fire-and-forget, but keep order via single queue)
```

16.7 CDN & Static Asset

- Frontend di-host terpisah (Vercel / Nginx static); backend cuma serve API.
- Attachment download URL adalah **presigned S3** — langsung dari CloudFront/MinIO, tidak via backend.
- Avatar (jika dipakai) di-cache 1 tahun dengan content hash di filename.

17. Security Requirements

17.1 Authentication

- **Password hashing:** BCrypt cost 12 (≈ 250 ms / hash di 2 vCPU). Jangan turunkan demi performance — itu yang melindungi DB dump.
- **JWT signing:** HS256 secret 256-bit dari env JWT_SECRET. Rotation tiap 90 hari; saat rotasi tetap accept token lama 24 jam.
- **Token transport:** Hanya via header Authorization. **Tidak** di-set sebagai cookie default — CSRF tidak relevan.
- **Brute force:** 5 failed login → lock email 15 menit. IP-level rate limit 20/menit.

17.2 Authorization (RBAC + Resource scoping)

```
// Method-level di service
@PreAuthorize("@accessChecker.canEditBoard(#boardId, principal)")
public BoardDto updateBoard(UUID boardId, ...) { ... }

// AccessChecker bean
public boolean canEditBoard(UUID boardId, AuthenticatedUser u) {
    // 1. Board ada di workspace yang user adalah member?
    // 2. Role member ∈ {Owner, Project Manager}?
    return membershipRepo.hasRole(boardId, u.id(), Set.of(OWNER, PM));
}
```

Role matrix

Action	Owner	PM	Designer	Developer	Other
Create board	Y	Y	N	N	N
Delete board	Y	N	N	N	N
Create / update card	Y	Y	Y	Y	Y (own)
Delete card	Y	Y	Y (own)	Y (own)	Y (own)
Move card	Y	Y	Y	Y	Y
Invite member	Y	Y	N	N	N
Change role	Y	N	N	N	N
Export finance	Y	Y	N	N	N
View finance	Y	Y	Y	Y	N

17.3 Input Validation

- Setiap DTO punya @NotBlank, @Size, @Email dll dari Jakarta Validation.
- Controller @Valid; pelanggaran → handler global → 400 dengan field detail.

- Path variable UUID di-parse via converter — invalid format → 400 sebelum controller.
- Query size di-clamp max 100 di service.

17.4 SQL Injection

- **Mandatory:** semua query lewat JPA / parameterized native query. Tidak ada string concat.
- Search field (LIKE) escape % dan _ di service.
- Sort field whitelist — terima nama logis (*value:desc*), bukan SQL column mentah.

17.5 XSS & Output Encoding

- API return JSON murni — XSS bukan attack-surface langsung.
- Komentar disimpan sebagai plain text. Frontend wajib escape saat render (React default).
- Filename attachment di-sanitize sebelum disimpan ke S3 (strip path traversal).

17.6 CORS

```
@Configuration
public class CorsConfig {
    @Bean public CorsConfigurationSource corsSource() {
        var c = new CorsConfiguration();
        c.setAllowedOrigins(List.of("https://app.promptara.id",
                                    "http://localhost:5173")); // dev only
        c.setAllowedMethods(List.of("GET", "POST", "PATCH", "PUT", "DELETE", "OPTIONS"));
        c.setAllowedHeaders(List.of("Authorization", "Content-Type", "If-Match"));
        c.setExposedHeaders(List.of("ETag", "X-Request-Id"));
        c.setAllowCredentials(false); // pakai bearer header, bukan cookie
        c.setMaxAge(3600L);
        var src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/api/**", c);
        return src;
    }
}
```

17.7 Rate Limiting

Bucket	Limit	Window	Scope
Login / Register	5	1 min	Per IP + per email
Default API	120	1 min	Per userId
File upload init	20	1 min	Per userId
Export endpoint	10	1 hour	Per userId

Implementasi pakai **Redis token bucket** (Bucket4j-Redisson) atau script Lua INCR + EXPIRE sederhana. Header response include X-RateLimit-Remaining dan Retry-After saat 429.

17.8 Secrets Management

- Semua secret (DB password, JWT secret, S3 key, Redis password) lewat env var; tidak hardcode.
- Local dev pakai .env yang di-gitignore.
- Production: AWS Secrets Manager / Vault → injected sebagai env saat container start.
- **Tidak pernah** log secret. Logback custom converter mask field sensitif.

17.9 OWASP Top 10 Checklist

Item	Mitigation
A01 Broken Access Control	RBAC + method-level @PreAuthorize + resource scoping
A02 Cryptographic Failures	BCrypt password, HS256 JWT, TLS at proxy
A03 Injection	JPA parameterized, no string concat, validation
A04 Insecure Design	Threat model dokumen ini; rate limit
A05 Security Misconfig	Spring profiles, no debug in prod, secure headers
A06 Vulnerable Components	Dependabot + monthly mvn versions:display
A07 Auth Failures	BCrypt + brute-force lockout + refresh rotation
A08 Data Integrity	Flyway checksums, signed JWT, optimistic lock
A09 Logging Failures	Structured logs, audit_log, 90 day retention
A10 SSRF	S3 client whitelist endpoint; no user-supplied URL fetch

18. Error Handling & Standard Response

18.1 Standard Error Envelope

```
{
  "timestamp": "2026-06-06T08:11:23.482Z",
  "status": 409,
  "code": "VERSION_CONFLICT",
  "message": "Card sudah diubah orang lain. Refresh dulu.",
  "path": "/api/v1/cards/01890d4c-...",
  "requestId": "req-9f1b...",
  "fields": [
    { "name": "email", "message": "format tidak valid" }
  ],
  "details": {
    "currentVersion": 7
  }
}
```

18.2 HTTP Status Mapping

Status	When	code field examples
400 Bad Request	DTO validation, malformed JSON	VALIDATION_ERROR, MALFORMED_JSON
401 Unauthorized	Missing/invalid/expired JWT	INVALID_CREDENTIALS, TOKEN_EXPIRED
403 Forbidden	Auth OK tapi role/scope kurang	FORBIDDEN, INSUFFICIENT_ROLE
404 Not Found	Resource tidak ada / soft-deleted	RESOURCE_NOT_FOUND
409 Conflict	Unique violation, version conflict	EMAIL_TAKEN, VERSION_CONFLICT
410 Gone	Token refresh sudah dipakai	REFRESH_TOKEN_REUSED
413 Payload Too Large	File > 50 MB	FILE_TOO_LARGE
422 Unprocessable	Business rule violation	DP_EXCEEDS_VALUE
429 Too Many	Rate limit	RATE_LIMITED
500 Internal	Unexpected	INTERNAL_ERROR
503 Service Unavailable	DB / Redis down	DEPENDENCY_DOWN

18.3 Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ErrorBody> onValidation(MethodArgumentNotValidException ex) {
        var fields = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> new ErrorField(f.getField(), f.getDefaultMessage()))
            .toList();
        return ResponseEntity.status(400)
    }
}
```

```
        .body(ErrorBody.of(400, "VALIDATION_ERROR", "Periksa input Anda.", fields));
    }

    @ExceptionHandler(OptimisticLockingFailureException.class)
    ResponseEntity<ErrorBody> onOptLock(OptimisticLockingFailureException ex) {
        return ResponseEntity.status(409)
            .body(ErrorBody.of(409, "VERSION_CONFLICT",
                "Data sudah diubah orang lain. Refresh dulu.));
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    ResponseEntity<ErrorBody> onIntegrity(DataIntegrityViolationException ex) {
        // Inspect SQL state – 23000 duplicate, 23503 FK violation, etc
        if (isUniqueViolation(ex)) {
            return ResponseEntity.status(409)
                .body(ErrorBody.of(409, "DUPLICATE", "Resource duplikat.));
        }
        return ResponseEntity.status(422)
            .body(ErrorBody.of(422, "INTEGRITY_ERROR", "Constraint violation.));
    }

    @ExceptionHandler(Exception.class)
    ResponseEntity<ErrorBody> onAny(Exception ex, HttpServletRequest req) {
        log.error("Unhandled at {}", req.getRequestURI(), ex);
        return ResponseEntity.status(500)
            .body(ErrorBody.of(500, "INTERNAL_ERROR",
                "Terjadi kesalahan. Tim sudah diberitahu.));
    }
}
```

18.4 Request Tracing

- Filter assign X-Request-Id (UUID) di awal request.
- Di-propagate ke MDC → semua log mencantumkan requestId.
- Header echoed kembali ke client untuk debugging cross-system.
- Honor X-Request-Id dari upstream jika ada (LB).

19. Observability — Logging, Metrics, Tracing

19.1 Structured Logging

```
# logback-spring.xml
<configuration>
  <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <customFields>{"app":"promptara-api","env":"${SPRING_PROFILES_ACTIVE}"}</customFields>
      <fieldNames>
        <timestamp>ts</timestamp>
        <message>msg</message>
        <level>lvl</level>
        <logger>logger</logger>
      </fieldNames>
    </encoder>
  </appender>
  <root level="INFO">
    <appender-ref ref="JSON"/>
  </root>
</configuration>
```

- Format JSON satu baris per event → mudah di-ingest Loki / ELK.
- MDC menyertakan: requestId, userId, workspaceId, path, method.
- Level **INFO** default; **DEBUG** hanya saat troubleshooting.
- Field sensitif (password, token, secret) di-mask via custom converter.
- Log retention 30 hari hot, 90 hari cold (S3 Glacier).

19.2 Metrics (Prometheus)

```
management:
  endpoints.web.exposure.include: health, info, prometheus, metrics
  endpoint.health.probes.enabled: true
  endpoint.prometheus.enabled: true
metrics:
  tags:
    app: promptara-api
  distribution:
    percentiles-histogram:
      http.server.requests: true
  slo:
    http.server.requests: 50ms, 100ms, 200ms, 500ms, 1s
```

Custom metrics

Metric	Type	Purpose
card_move_total	Counter	Volume drag-drop
card_optimistic_conflict_total	Counter	Lock contention warning
login_failed_total	Counter	Brute force watcher

Metric	Type	Purpose
board_cache_hit_ratio	Gauge	Tune TTL
attachment_upload_bytes	Summary	Bandwidth
notification_fanout_seconds	Timer	Async health
db_pool_active	Gauge (HikariCP)	Capacity

19.3 Health Endpoints

```
GET /actuator/health/liveness
→ 200 jika app process alive (always)

GET /actuator/health/readiness
→ 200 jika: DB reachable + Redis reachable + Flyway done
→ 503 jika ada dependency down – load balancer remove dari rotation
```

19.4 Distributed Tracing (opsional V1, wajib V2)

- Micrometer Tracing + OTLP exporter → Tempo / Jaeger.
- Trace propagation: traceparent header (W3C).
- Sample rate 10% di production, 100% di staging.

20. Docker, Docker-Compose & Deployment

20.1 Dockerfile (multi-stage)

```
# syntax=docker/dockerfile:1.7
# ■■■ Stage 1: build
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /workspace
COPY .mvn .mvn
COPY mvnw pom.xml ./
RUN ./mvnw -q dependency:go-offline
COPY src src
RUN ./mvnw -q -DskipTests package && \
    mkdir -p target/extracted && \
    java -Djarmode=layertools -jar target/*.jar extract --destination target/extracted

# ■■■ Stage 2: runtime
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S app && adduser -S app -G app && \
    apk add --no-cache curl tzdata
ENV TZ=Asia/Jakarta \
    JAVA_OPTS="-XX:MaxRAMPercentage=70 -XX:+UseG1GC -XX:+ExitOnOutOfMemoryError"
WORKDIR /app
COPY --from=build /workspace/target/extracted/dependencies/ ./
COPY --from=build /workspace/target/extracted/spring-boot-loader/ ./
COPY --from=build /workspace/target/extracted/snapshot-dependencies/ ./
COPY --from=build /workspace/target/extracted/application/ ./
USER app
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD curl -fsS http://localhost:8080/actuator/health/liveness || exit 1
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS org.springframework.boot.loader.launch.JarLauncher
"]
```

20.2 docker-compose.yml (development)

```
version: "3.9"
services:

  mysql:
    image: mysql:8.0
    container_name: promptara-mysql
    restart: unless-stopped
    command: >
      --default-authentication-plugin=caching_sha2_password
      --character-set-server=utf8mb4
      --collation-server=utf8mb4_0900_ai_ci
      --innodb-buffer-pool-size=512M
      --innodb-lock-wait-timeout=5
      --max-connections=200
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: promptara
      MYSQL_USER: promptara
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    ports: [ "3307:3306" ]
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
```

```
test: ["CMD", "mysqladmin", "ping", "-h", "localhost",
      "-u", "promptara", "-p${MYSQL_PASSWORD}"]
interval: 10s
timeout: 3s
retries: 12

redis:
  image: redis:7-alpine
  container_name: promptara-redis
  restart: unless-stopped
  command: ["redis-server", "--requirepass", "${REDIS_PASSWORD}",
            "--maxmemory", "256mb", "--maxmemory-policy", "allkeys-lru"]
  ports: [ "6380:6379" ]
  volumes: [ "redis-data:/data" ]
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 10s

minio:
  image: quay.io/minio/minio:latest
  container_name: promptara-minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ROOT_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
  ports:
    - "9000:9000"      # S3 API
    - "9001:9001"      # Web console
  volumes: [ "minio-data:/data" ]

adminer:
  image: adminer:4-standalone
  container_name: promptara-adminer
  restart: unless-stopped
  ports: [ "8081:8080" ]
  depends_on: [ mysql ]

api:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: promptara-api
  restart: unless-stopped
  depends_on:
    mysql:
      condition: service_healthy
    redis:
      condition: service_healthy
    minio:
      condition: service_started
  environment:
    SPRING_PROFILES_ACTIVE: docker
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/promptara?useSSL=false&serverTimezone=UTC
    SPRING_DATASOURCE_USERNAME: promptara
    SPRING_DATASOURCE_PASSWORD: ${MYSQL_PASSWORD}
    SPRING_DATA_REDIS_HOST: redis
    SPRING_DATA_REDIS_PORT: 6379
    SPRING_DATA_REDIS_PASSWORD: ${REDIS_PASSWORD}
    JWT_SECRET: ${JWT_SECRET}
    S3_ENDPOINT: http://minio:9000
    S3_BUCKET: promptara-attach
    S3_ACCESS_KEY: ${MINIO_ROOT_USER}
```

```
S3_SECRET_KEY: ${MINIO_ROOT_PASSWORD}
ports: [ "8080:8080" ]
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health/readiness"]
  interval: 15s
  timeout: 3s
  retries: 5
  start_period: 60s

volumes:
  mysql-data:
  redis-data:
  minio-data:
```

20.3 .env.example

```
MYSQL_ROOT_PASSWORD=change-me-root
MYSQL_PASSWORD=change-me-app
REDIS_PASSWORD=change-me-redis
MINIO_ROOT_USER=promptara-minio
MINIO_ROOT_PASSWORD=change-me-minio
JWT_SECRET=64-char-random-secret-base64-encoded-please-change-me-now-thanks
```

20.4 Local development workflow

```
# clone & setup
cp .env.example .env          # isi nilai secret
docker compose up -d mysql redis minio
./mvnw spring-boot:run -Dspring-boot.run.profiles=local

# or full stack in containers
docker compose up -d
docker compose logs -f api

# down + wipe volumes (CAREFUL — drops DB)
docker compose down -v
```

20.5 Production deployment notes

- Production tidak pakai compose; pakai orchestrator (ECS / Kubernetes / Nomad).
- MySQL pakai managed service (RDS / CloudSQL); jangan jalankan di container.
- Redis pakai managed service (ElastiCache); enable persistence + backup.
- Image API di-push ke registry (ECR / GHCR); rolling deploy via blue-green.
- Backup: mysqldump --single-transaction daily + binlog continuous.
- Secret injection lewat AWS Secrets Manager / Kubernetes Secret.

21. Spring Boot Project Structure

21.1 Recommended Package Layout

```

src/main/java/id/promptara/api/
■■■■ PromptaraApplication.java           ← main class
■■■■ common/
■ ■■■■ config/                          ← @Configuration beans
■ ■ ■■■ SecurityConfig.java
■ ■ ■■■ CorsConfig.java
■ ■ ■■■ RedisConfig.java
■ ■ ■■■ JpaConfig.java
■ ■ ■■■ OpenApiConfig.java
■ ■ ■■■ AsyncConfig.java
■ ■ ■■■ CacheConfig.java
■ ■■■■ exception/
■ ■ ■■■ GlobalExceptionHandler.java
■ ■ ■■■ ErrorBody.java
■ ■ ■■■ NotFoundException.java
■ ■ ■■■ ConflictException.java
■ ■ ■■■ ...
■ ■■■■ security/
■ ■ ■■■ JwtFilter.java
■ ■ ■■■ JwtService.java
■ ■ ■■■ AuthenticatedUser.java
■ ■ ■■■ AccessChecker.java
■ ■■■■ ratelimit/
■ ■ ■■■ RedisRateLimiter.java
■ ■■■■ util/
■ ■ ■■■ UuidV7Generator.java
■ ■ ■■■ ...
■■■■ auth/
■ ■■■ api/AuthController.java
■ ■■■ api/dto/                          ← LoginRequest, RegisterRequest, AuthResponse
■ ■■■ service/AuthService.java
■ ■■■ service/RefreshTokenService.java
■ ■■■ persistence/
■ ■ ■■■ RefreshTokenEntity.java
■ ■ ■■■ RefreshTokenRepository.java
■■■■ user/
■ ■■■ api/...
■ ■■■ service/UserService.java
■ ■■■ persistence/
■ ■ ■■■ UserEntity.java
■ ■ ■■■ UserRepository.java
■■■■ workspace/
■ ■■■ ...
■■■■ board/
■ ■■■ api/BoardController.java
■ ■■■ api/ColumnController.java
■ ■■■ service/...
■ ■■■ persistence/...
■■■■ card/
■ ■■■ api/CardController.java
■ ■■■ api/dto/...

```

```

■   ■■■ service/
■   ■   ■■■ CardService.java
■   ■   ■■■ CardMoveService.java
■   ■   ■■■ CardOrderingStrategy.java
■   ■■■ persistence/
■       ■■■ CardEntity.java
■       ■■■ CardRepository.java
■
■■■ comment/...
■■■ attachment/...
■■■ notification/
■   ■■■ api/NotificationController.java
■   ■■■ service/
■       ■■■ NotificationService.java
■       ■■■ NotificationFanoutListener.java
■       ■■■ DueSoonScheduler.java
■   ■■■ persistence/...
■
■■■ finance/
■   ■■■ api/FinanceController.java
■   ■■■ api/OrderController.java
■   ■■■ service/
■       ■■■ FinanceQueryService.java    ← native query
■       ■■■ OrderExportService.java    ← streaming CSV
■
■■■ audit/
■   ■■■ service/AuditService.java
■   ■■■ persistence/AuditLogEntity.java

src/main/resources/
■■■ application.yml
■■■ application-docker.yml
■■■ application-prod.yml
■■■ db/migration/V*.sql
■■■ logback-spring.xml

src/test/java/id/promptara/api/
■■■ auth/...      (unit)
■■■ card/...      (unit + integration with Testcontainers)
■■■ integration/... (full Spring slice tests)

```

21.2 Coding Conventions

- Controller **tipis**: validasi DTO, panggil service, map ke response. Tidak ada business logic.
- Service mengandung **satu transaction boundary** per public method.
- Repository **satu** per aggregate root; tidak return entity ke controller.
- DTO request/response **terpisah** dari entity. Mapper via MapStruct.
- Constructor injection (final field); tidak @Autowired field.
- Method publik service punya Javadoc *singkat*: input, output, side effect.

22. Implementation Roadmap & Milestones

22.1 Milestones (Estimasi 8 minggu, 2 engineer)

Sprint	Week	Scope	Deliverable
M1: Foundation	1-2	Project skeleton, Docker stack, Flyway V1-V4, Auth (login/register/reset/swap JWT), RBAC dasar	Length of sprint: 2 weeks
M2: Kanban core	3-4	Board CRUD, Column CRUD + reorder, Card CRUD, Content Board, New issue, and attachments	Content Board view bisa scroll dan optimalisasi Storage ke S3
M3: Collaboration	5	Comments, attachments (presigned), notifications fan-out, modal logs fully functional	Card modal logs
M4: Read-side	6	Orders list dengan filter, Finance aggregation + cache, Order View, Finance View dipindah ke API	Order View, Finance View
M5: Hardening	7	Rate limit, structured logging, metrics, healthchecks, Coverage ≥ 95%, Test container, load test passes	Deployment to production
M6: Pre-launch	8	Penetration test fix, performance tuning, runbook, monitoring dashboard	Production goes live

22.2 Critical Dependencies

- **S3-compatible storage** (MinIO local, AWS S3 prod) sebelum M3.
- **SMTP relay** (Mailgun / SES) jika email verification dinyalakan — opsional V1.
- **Reverse proxy** (Nginx) untuk TLS sebelum production go-live.
- **Monitoring stack** (Prometheus + Grafana + Loki) sebelum M6.

22.3 Risks & Mitigations

Risk	Impact	Mitigation
MySQL lock contention saat drag-drop	High	Sparse order + load test sejak M2; siapkan repack cron
File upload memory pressure	Medium	Presigned URL — backend tidak handle byte; dites M3
Refresh token theft	High	Rotation + reuse detection; alert otomatis
Flyway migration gagal di prod	High	CI run terhadap staging snapshot; never edit V file
Cache stampede di finance summary	Medium	TTL jitter + single-flight lock (Redis SETNX) saat refresh
Schema drift Entity vs DDL	Medium	Test ddl-auto=validate di CI

23. Appendix — Acceptance Criteria & Test Plan

23.1 Acceptance Criteria (sample for Auth)

- GIVEN email belum terdaftar, WHEN POST /auth/register dengan body valid, THEN 201 + user + accessToken + refreshToken.
- GIVEN email sudah terdaftar, WHEN POST /auth/register, THEN 409 dengan code EMAIL_TAKEN.
- GIVEN 5 login fail dalam 10 menit, WHEN login ke-6, THEN 429 selama 15 menit walaupun password benar.
- GIVEN refresh token valid, WHEN POST /auth/refresh dua kali bersamaan, THEN tepat satu berhasil (200), satu gagal (401).
- GIVEN access token valid, WHEN logout, THEN access token langsung ditolak di endpoint selanjutnya.

23.2 Acceptance Criteria (Card)

- User A dan B GET card v=3 bersamaan; A PATCH (If-Match: 3) sukses (v→4); B PATCH (If-Match: 3) gagal 409 dengan currentVersion=4.
- User A dan B POST /cards/X/move bersamaan ke kolom berbeda; akhir state: card ada di satu kolom saja, version increment 2.
- DELETE card → tidak muncul di GET board; restore dalam 30 hari → muncul lagi dengan version yang sama.
- POST card dengan dpPaidIdr > valueIdr*1.1 → 422 dengan code DP_EXCEEDS_VALUE.

23.3 Test Pyramid

Layer	Tooling	Coverage Target
Unit (service/util)	JUnit 5 + Mockito	≥ 80%
Integration (slice)	@DataJpaTest + Testcontainers MySQL Repository + JPQL	
Integration (full)	@SpringBootTest + Testcontainers (mysql, kafka, redis)	Happy + edge per endpoint
Contract	REST Assured against running container	All endpoints
Load	k6 / Gatling	Board GET 500 RPS, p95 < 200ms
Security	OWASP ZAP baseline	Pass CI gate

23.4 Load Test Scenarios

- **Board read storm:** 200 user, 5 board per user, 2 req/s GET board → 60s, target p95 < 200 ms.

- **Drag-drop storm:** 50 user di 1 board, move random card setiap 1-3 detik → tidak boleh ada 409 spike, p95 < 250 ms.
- **Finance summary:** 50 user spam GET finance summary → cache hit ratio > 90%, p95 < 100 ms.
- **Export CSV:** 10 user trigger export ribuan row → memori tidak naik > 200 MB.

23.5 Pre-Production Checklist

- All Flyway migration tested terhadap staging restore.
- `ddl-auto`: `validate` aktif di production profile.
- `flyway.clean-disabled`: `true` di semua environment.
- Tidak ada `System.out.println` sisa.
- `SQL_MODE` MySQL: `STRICT_TRANS_TABLES`, `NO_ZERO_DATE`, `NO_ENGINE_SUBSTITUTION`.
- TLS termination di proxy, HSTS header aktif.
- Backup MySQL otomatis terverifikasi (test restore minimal sekali).
- Runbook untuk: deadlock storm, DB down, Redis down, attachment storage down.
- On-call rotation dijadwalkan.

— END OF DOCUMENT —

PROMPTARA Backend PRD v1.0 · Spring Boot · MySQL · Flyway · Docker
Dokumen ini hidup; kirim PR untuk koreksi/usulan ke repo backend.